

Deployment Strategy Report for Email Triage AI Model

Keith G. Broussard

University of the Cumberland

MSAI-699-B01: Capstone

Dr. Gamini Bulumulle

August 2, 2026

Deployment Strategy Report for Email Triage AI Model

Current Architecture and Deployment Recommendation

The project is a scikit-learn binary urgency classifier that combines email subject and body text, creates sparse TF-IDF features, and applies class-balanced logistic regression. The retained Week 4 candidate uses $C = 0.1$, unigrams, $\text{min_df} = 2$, and sublinear term frequency, producing 2,895 features. It was developed from 199 reviewed Enron messages, only 30 of which were urgent. Mean five-fold cross-validation urgent F1 was 11.4%, while the preliminary holdout retained four urgent false negatives. These results support refinement, not automated email handling (Broussard, 2026). The repository also lacks a serialized serving artifact, API, and deployment benchmarks.

Cloud deployment would centralize versions, monitoring, and elastic scaling, but it would transmit sensitive email content and require provider, identity, cost, and compliance controls. Edge deployment would run on a trusted workstation or managed internal host near the email source. It limits data movement and suits sparse linear inference, although updates, capacity, and monitoring become local responsibilities. Course materials emphasize defining performance, compute, data, and serving constraints before testing assumptions in real use (Huyen, n.d.; G. Bulumulle, personal communication, July 31, 2026).

I recommend an edge-first, CPU-based shadow pilot. This is a proposal, not a verified deployment. The vectorizer-classifier pipeline should be persisted from a trusted build, packaged with pinned dependencies, and loaded on a controlled host. The architecture does not justify a GPU. Predictions would appear only in the review queue; the service would not move, delete, flag, or respond to emails. The assigned video also recommends stable endpoints, monitoring, and shadow evaluation before users depend on a model (Dutrée, 2023).

FastAPI Integration, Scalability, and Efficiency

External FastAPI documentation supports this choice: FastAPI fits the Python project, validates typed request bodies, and generates an OpenAPI contract (FastAPI, n.d.). A versioned POST /v1/triage endpoint would accept subject and body strings with length limits. It would return the predicted label, the urgent probability, a needs_human_review flag, the model version, and the contributing terms. A GET /health endpoint would report readiness without exposing message data. The fitted pipeline would load once at startup. An authorized email client would call the internal endpoint, and its response would go only to a human review interface.

Model persistence is a deployment gate. External scikit-learn documentation warns that pickle-based formats can execute untrusted code and cross-version loading is unsupported. The artifact, training recipe, dependency versions, and data snapshot, therefore, need recorded provenance (Scikit-learn developers, n.d.). A format such as skops.io can be evaluated, but it must be tested with this pipeline. Routine logs should exclude subjects and bodies while retaining request identifiers, model version, response time, status, and aggregate outcomes.

The 2,895-feature space suggests efficient CPU inference, but Week 4 timing was not a serving benchmark. The pilot should measure artifact size, startup time, memory per worker, p50 and p95 latency, concurrent throughput, and errors. Begin with one worker and a queue limit, then scale only after measured demand. Rate limits, timeouts, payload limits, and backpressure protect availability. If broader access is later justified, the same container and stable contract could move to a private cloud. This follows the course-provided API reading on triggers, connectors, monitoring messages, and centralized control (Bricks Tech, 2025).

Ethics, Security, and Deployment Gates

The Enron sample is small, urgency-enriched, and organization-specific. It has no reliably protected demographic attributes, so demographic fairness cannot be claimed. The review should instead compare false negatives, false positives, overrides, and confidence across the available message characteristics. The central harm is a missed urgent message: the preliminary holdout missed four of six. The threshold should not change until new validation data support the tradeoff. Top linear feature contributions can assist review, while SHAP remains an offline audit tool; neither proves why a message is urgent.

Email requires least-privilege authentication and authorization, encrypted transport, secrets outside the source code, minimal retention, and restricted administrative endpoints. Validation must reject oversized or malformed requests. External OWASP guidance recommends rate limits, timeouts, bounded concurrency, and resource alerts because unrestricted API consumption can lead to denial-of-service attacks and unexpected costs (OWASP Foundation, 2023). Logs and explanations should exclude full text, addresses, and unnecessary identifiers. The service should load only trusted, checksummed model artifacts.

Deployment should pass four gates: reproduce the pipeline; validate new held-out data; run a human-reviewed shadow pilot; and review urgent recall, privacy, security, latency, and capacity. Monitor performance, reviewer disagreement, drift, errors, and versions. The external NIST AI Risk Management Framework treats validity, security, privacy, transparency, and harmful bias as continuing responsibilities (Tabassi, 2023). An edge-first shadow service is feasible under these controls; automation or cloud expansion remains conditional on stronger evidence and verified requirements. The Appendix summarizes the proposed workflow.

References

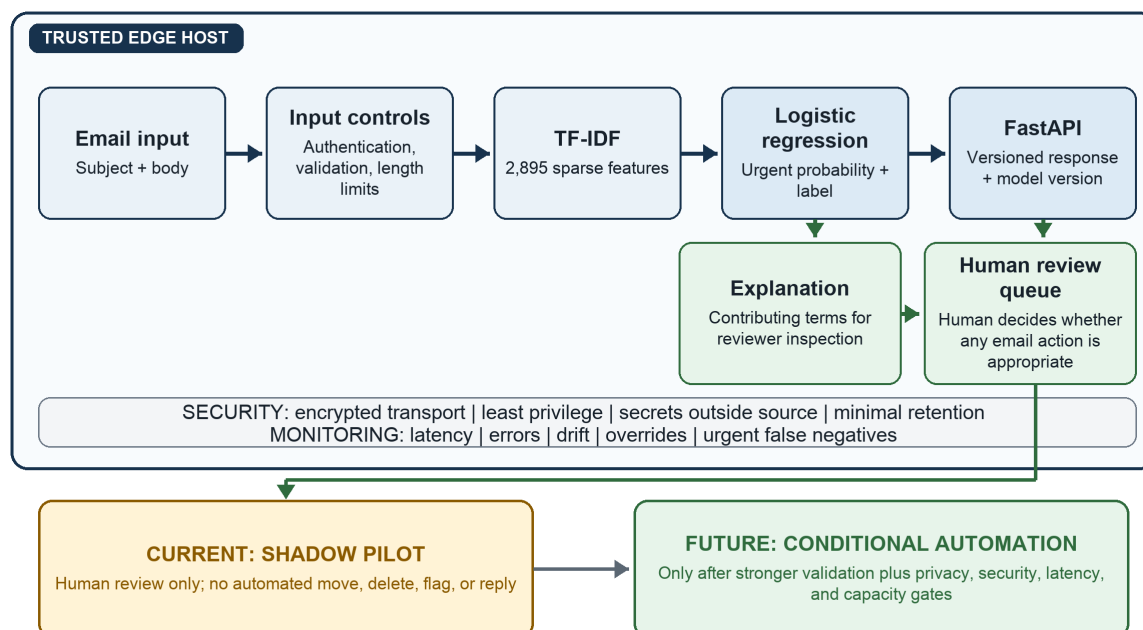
- Bricks Tech. (2025, August 18). *A comprehensive guide to API integration in 2025*.
<https://www.brickstech.io/blogs/a-comprehensive-guide-to-api-integration-in-2025>
- Broussard, K. G. (2026). *Week 4 model optimization report* [Unpublished manuscript]—
 University of the Cumberland.
- Dutrée, A. (2023, November 10). *Deploying generative AI models: Best practices and an interactive example* [Video]. Toronto Machine Learning Society. YouTube.
https://www.youtube.com/watch?v=BvA9b39a_ag
- FastAPI. (n.d.). *Request body*. Retrieved July 31, 2026, from
<https://fastapi.tiangolo.com/tutorial/body/>
- Huyen, C. (n.d.). *Machine learning systems design*. <https://huyenchip.com/machine-learning-systems-design/toc.html>
- OWASP Foundation. (2023). *OWASP API Security Top 10 - 2023*. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- Scikit-learn developers. (n.d.). *Model persistence*. Retrieved July 31, 2026, from https://scikit-learn.org/stable/model_persistence.html
- Tabassi, E. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.AI.100-1>

Appendix

Proposed Edge-First Deployment and Human-Review Workflow

Figure A1

Explainable Edge Deployment and Human-Review Workflow



Current evidence. The reviewed data contain 199 messages, including 30 urgent messages. The retained pipeline uses 2,895 TF-IDF features, achieved 11.4% mean cross-validation urgent F1, and retained four urgent false negatives in the preliminary holdout (Broussard, 2026).

Note. This author-created figure presents a proposed architecture, not an implemented or benchmarked system. Human review remains required because the current model has a substantial risk of false negatives for urgent messages. Automation or private cloud expansion remains contingent on stronger validation and verified operational requirements.